

Tyler Vander Mooren

CYB 499: LLM and Cyber Security Applications

Professor Hayretin Bahsi

March 18, 2026

Homework Assignment #2

Question 1: Impact of Prompting Techniques on LLM Performance for Cybersecurity

Multiple-Choice Questions

Setup

A set of 30 cybersecurity multiple-choice questions were reused from Assignment #1.

1. Question Set: questions.txt
2. Answer Key: answersforquestions.txt

Topics:

- Threat modeling (STRIDE, DREAD)
- Attack surface measurement
- Security reviews
- Secure Development Lifecycle
- Integrity, availability and non-repudiation

To properly evaluate few-shot prompting without data leakage, 10 questions were used as demonstration examples, while the remaining 20 questions were used for the scoring.

Models:

1. GPT-5-nano via OpenAI API (~8-18 billion parameters, instruction-tuned model)
2. GPT-2 XL via HuggingFace Local (1.5 billion parameters, non-instruction tuned model)

Each model received the same set of evaluation questions under multiple prompting strategies.

The prompting techniques that were tested:

- Zero-shot prompting
- Role assignment prompting
- Few-shot prompting (2-shot, 5-shot, 10-shot)
- Sample order variation (5-shot reversed)
- Self-consistency prompting (majority vote across 5 runs)

Responses were recorded and compared against the answer key to compute accuracy.

Results

Table 1

Model Performance across Prompting Techniques

Model	Condition	Correct	Accuracy
GPT-5-nano	Zero-shot	15/20	75%
GPT-5-nano	Role	15/20	75%
GPT-5-nano	2-shot	15/20	75%
GPT-5-nano	5-shot	15/20	75%
GPT-5-nano	10-shot	15/20	75%
GPT-5-nano	5-shot (reversed)	15/20	75%
GPT-5-nano	Self-consistency	15/20	75%
GPT-2 XL	Zero-shot	2/20	10%
GPT-2 XL	Role	2/20	10%
GPT-2 XL	2-shot	4/20	20%
GPT-2 XL	5-shot	5/20	25%
GPT-2 XL	10-shot	3/20	15%
GPT-2 XL	5-shot (reversed)	7/20	35%
GPT-2 XL	Self-consistency	2/20	10%

Error Analysis and Prompting Evaluation

1. Role Assignment Prompting

For role assignment prompting the model was instructed to act as a senior cybersecurity analyst with expertise in secure software development, threat modeling and SDL processes. For

GPT-5-nano, this produced no measurable change and accuracy remained at 75%. That suggests the model handled the task within an instruction-aligned framework and did not need additional framing to answer the structured multiple-choice questions. For example, Question 20 GPT-5-nano selected c under both zero-shot and role prompting and both answers were correct. GPT-2 XL also showed no improvement under role prompting and remained at 10% accuracy. It did not appear to use the role definition at all and often fell back on random or patterned responses. In short, although role-prompting did not help either model, it did so for different reasons. In the case of GPT-5-nano it simply did not need it, while for GPT-2 XL it could not reliably use it.

2. Zero-shot vs Few-shot Prompting

Few-shot prompting was done by adding labeled examples before each question using 2-shot, 5-shot and 10-shot configurations. For GPT-5-nano the performance stayed fixed at 75% across zero-shot, 2-shot, 5-shot and 10-shot prompting. That suggests the added examples did not improve its decision-making for this task. Question 11 illustrates the pattern well and GPT-5-nano answered b correctly under every configuration with no change in output. For GPT-2 XL, few-shot prompting produced only limited and inconsistent gains:

- 2-shot: 20%
- 5-shot: 25%
- 10-shot: 15%

Both 2-shot and 5-shot improved on the 10% zero-shot baseline, but 10-shot went down to 15%. This suggests that adding more examples did not steadily improve performance and may have introduced noise instead. Question 20 shows this instability clearly and under zero-shot prompting, GPT-2 XL selected a, which was incorrect. Under 2-shot prompting, it changed to c, which was correct. Under 5-shot prompting, it shifted to d, which was incorrect and under 10-

shot prompting it returned to c, which was correct. Overall, few-shot prompting gave GPT-2 XL only minor gains and had no measurable effect on GPT-5-nano.

3. Sample Order in Few-Shot Prompting

To test order sensitivity, the 5-shot prompt examples were reversed. For GPT-5-nano, total performance stayed at 75%, but some individual answers changed. In Question 20, the original 5-shot prompt produced the correct answer c, while the reversed 5-shot prompt changed the answer to a. In Question 21, the opposite happened and the original 5-shot prompt produced a, while the reversed order produced b. So even though the final score did not change, example order still influenced specific outputs.

GPT-2 XL was much more sensitive to example order. Its accuracy increased from 25% under the original 5-shot condition to 35% under the reversed 5-shot condition. Questions 26, 28, and 30 show the clearest pattern. In the original 5-shot condition, GPT-2 XL answered all three as c, which is incorrect. After reversing the examples, it changed all three answers to d, which is correct. This seems to point to recency bias, where the examples later in the prompt had a stronger effect on the model's final choice.

4. Self-Consistency Prompting

To test self-consistency prompting the model was asked each question five times and then the majority answer was selected. For GPT-5-nano, this produced no improvement and accuracy remained at 75%. The model was already showing to be highly stable across repeated runs, so there was little variation for majority voting to fix. Question 21 is the best example of this. Across five runs, GPT-5-nano produced the results a, a, a, a and b. So, even though one run produced the correct answer of b, the majority answer remained incorrect with a.

GPT-2 XL also showed no improvement under self-consistency and remained at 10%. Its output was generally repetitive rather than being variable. The output for Question 23 shows this with GPT-2 XL producing the correct answer a in all five runs, but while on other questions it would repeat the same incorrect answer just as consistently. Since majority voting only helps when there is enough variation, self-consistency was ineffective for this model as well.

Model Output Comparison

The results show that prompting techniques affect models differently depending on architecture and alignment. GPT-5-nano, as an instruction-tuned model, already operates within a structured response framework. Its outputs were stable, consistent and followed task constraints regardless of the prompt variation. At the same time, its unchanged 75% accuracy across every condition suggests prompt engineering did not improve its underlying reasoning on this task. It appears to have reached its operating performance ceiling here.

GPT-2 XL, by contrast lacks instruction tuning and did not naturally follow structured task constraints. Few-shot prompting provided some guidance and led to small gains in a few conditions, but those gains were unstable and highly sensitive to prompt design, especially the example order. The model did not show reliable reasoning or consistent formatting and the prompting changes did not overcome its deeper architectural limitations. Overall, the results suggest that prompting mainly changes task framing and response behavior, but it cannot change core reasoning capability.

Analysis

Several conclusions can be drawn from these results. One is that prompting techniques do not universally improve model performance. Their effectiveness depends heavily on the underlying model. The instruction-tuned model GPT-5-nano remained stable across all

prompting strategies, which suggests it already internalizes the task structure without needing much extra guidance. GPT-2 XL, on the other hand, showed only minor and inconsistent gains, along with much stronger sensitivity to example order. Also, increasing prompt complexity does not necessarily improve results. GPT-2 XL rose from 10% in zero-shot prompting to 25% with 5-shot prompting but then it dropped to 15% with 10-shot prompting, which suggests that extra examples can add noise instead of clarity for weaker models. Furthermore, self-consistency prompting did not improve either of these models, showing that majority voting only helps when the model produces enough useful variation for the vote to correct an error.

Overall, the effectiveness of prompting techniques was strongly dependent on the model itself. GPT-5-nano maintained consistent performance regardless of prompting strategy but did not gain anything meaningful from additional prompt engineering. GPT-2 XL showed some improvement under a few few-shot conditions, yet its overall performance remained far below acceptable levels and varied noticeably with prompt design. These results suggest that prompt engineering works better as a refinement tool than as a fix. It can help when a model already has enough reasoning capability, but it cannot overcome deeper architectural or alignment limitations. For structured cybersecurity evaluation tasks, model selection remains the most important factor, while prompting will provide only marginal benefits.